CIS 568 · DATA VISUALIZATION · FINAL PROJECT

# Interactive Visualization of NTSB Aviation Accident Data

*A Browser-Based Platform for Exploring Six Decades of Civil Aviation Safety Data*

---

**Michael Evan**

Graduate Computer Science Department
University of Massachusetts Dartmouth

SPRING 2026

## Introduction

Aviation safety data collected by the National Transportation Safety Board (NTSB) spans over six decades of US civil aviation accidents. The database contains detailed records including aircraft information, injury severity, weather conditions, probable cause narratives, and geographic coordinates for each event. While this data is publicly available, exploring it in its raw form offers little insight without the ability to filter, compare, and visualize patterns across multiple dimensions simultaneously. This project builds an interactive, browser-based visualization platform that enables real-time exploration of 168,793 NTSB aviation accident records from 1962 to 2025, combining geographic mapping with analytical charts and multi-dimensional filtering to surface patterns in the data that static reports cannot reveal.

## Problem Definition

The NTSB Aviation Accident Database is one of the most comprehensive aviation safety datasets in the world, yet accessing and analyzing it remains difficult for most users. The raw data is delivered as flat CSV files with inconsistent formatting across decades of data entry by different investigators. Many records lack precise coordinates, aircraft field naming conventions changed over the years, and multi-aircraft events are stored as comma-separated values within single fields rather than normalized across rows. Building a visualization tool on this data required not only the technical implementation of interactive maps and charts but also significant data cleaning and enrichment to make the records usable. The challenge was to build a single-page application capable of rendering 168,793 records across multiple visualization modes with real-time filtering, without requiring any server-side infrastructure.

## Methodology

The project was built as a single HTML file with all functionality implemented client-side using JavaScript. The technology stack was selected based on the strengths of each library for specific visualization tasks:

- **Leaflet** was chosen for the interactive map because it handles tile-based maps with pan, zoom, and marker rendering efficiently. Leaflet's plugin ecosystem provided MarkerCluster for grouping nearby accidents and Leaflet.heat for density visualization.
- **D3.js** was chosen for all analytical charts because it provides full control over SVG and canvas rendering. D3 powers the line chart (accidents per year), horizontal bar chart (top aircraft makes), donut chart (injury severity breakdown), choropleth (state-level accident density), sparklines (trend indicators), and the full-screen spike map with county-level drill-down.
- **TopoJSON** provided the geographic boundary data for US states and counties, enabling the choropleth and spike map visualizations.
- **PapaParse** handled client-side CSV parsing of the 168,793-record dataset.
- **noUiSlider** provided the dual-handle year range slider for date filtering.

All basemap tiles are served from external tile providers. CARTO provides the dark and street map tiles, the FAA provides VFR sectional chart tiles, and OpenTopoMap provides terrain/elevation tiles. The application requires no backend server and runs entirely in the browser, making it suitable for deployment on GitHub Pages as a static site.

## Implementation

## Data Preprocessing and Enrichment

The raw NTSB dataset required significant cleaning before it could be used effectively in the visualization. The following preprocessing steps were performed offline using Python 3 with Pandas for data manipulation, CSV parsing, and analysis, and Shapely for geographic point-in-polygon computations:

**Coordinate Recovery:** A substantial number of older accident records lacked precise GPS coordinates. Using airport identifier codes and city/state information from the records, coordinates were recovered by cross-referencing against a database of 32,452 US airports and a city centroid lookup table. Each record's coordinate source was tagged (GPS/survey, airport lookup, city centroid, or state centroid) to maintain transparency about data quality. Records with city-level coordinates were flagged in the application's popup display with a warning indicator.

**Coordinate Jitter:** Many accidents geocoded to city centroids shared identical coordinates, causing dots to stack on top of each other on the map. A random offset of approximately 1km was applied to city and state centroid records so individual accidents spread apart when zoomed in while still appearing clustered at the city level when zoomed out.

**Engine Count Correction:** The NumberOfEngines field contained numerous blanks and errors. Gliders and balloons with blank engine counts were set to 0. Helicopters and airplanes with missing counts were identified by Make and Model and assigned the correct engine count. Known single-engine models (Bell 206, Cessna 172, etc.) were set to 1, and known twin-engine models (Sikorsky S-76, Bell 412, Boeing 737, etc.) were set to 2 or 4 as appropriate. Individual records were verified against publicly available aircraft specifications.

**Multi-Aircraft Flagging:** Multi-aircraft accidents (midair collisions, runway incursions, ground collisions) were identified using two methods. First, records with comma-separated values in the AirCraftCategory field indicated two aircraft were involved in the same event. Second, the ProbableCause text was searched for keywords including “midair collision,” “other aircraft,” “runway incursion,” and similar phrases. Both methods were combined and the results were manually reviewed to remove false positives such as bird strikes, tree collisions, and other single-aircraft events that happened to mention another aircraft in the narrative. A total of 2,124 multi-aircraft accidents were identified and flagged.

**County Assignment:** Each accident record was assigned to a US county using point-in-polygon geometric testing against 3,220 county boundary polygons. A bounding box pre-filter was applied to reduce the computational cost by eliminating county polygons that could not possibly contain a given point before running the full geometry check. County FIPS codes and names were stored directly in the CSV to eliminate any runtime geometry calculations. The Python Shapely library with prepared geometries provided the spatial analysis.

**State Centroid Records:** Approximately 14,658 records had coordinates placed at their state's geographic centroid due to insufficient location data in the original NTSB records. These records were merged into the main dataset with all enrichments applied but are identified separately in the application. Their data contributes to all statistical calculations but their map positions are acknowledged as approximate through “unlocated” indicators in the spike map and county drill-down views.

## Map Visualization Modes

The application provides four basemap options and three interactive map modes for viewing accident data on the Leaflet map:

**Cluster Mode** groups nearby accident markers into numbered circles that split apart as the user zooms in. Cluster icons are color-coded by the worst injury severity in the group (red for fatal, orange for serious, yellow for minor, green for none). Individual markers use DOM-based divIcon elements to ensure reliable click interaction for opening accident detail popups. A batched rendering system with a progress indicator prevents the browser from freezing during the creation of 168,793 markers, and mode-switching buttons are disabled during rendering to prevent users from interrupting the process.

**Heatmap Mode** renders accident density as a continuous color gradient using the Leaflet.heat plugin. Intensity is weighted by injury severity so fatal accidents contribute more visual weight than minor incidents.

**All Dots Mode** renders every accident as an individual circle marker on an HTML5 canvas element for performance. A batched rendering system with a token-based cancellation mechanism prevents old render batches from contaminating new renders when filters change mid-build. Dot size scales with zoom level for visibility at different scales.

**Timeline Mode** animates through the dataset year by year, re-rendering the current map mode for each year's data with a playback transport bar showing year, progress, and play/pause controls.

### Spike Map and County Drill-Down

A full-screen D3.js spike map provides a state-level overview where vertical spikes rise from each state proportional to accident count. Spike color transitions from blue (low) through cyan (mid) to red (high). Hovering over any state or spike displays a tooltip with accident count, fatal count, fatality rate, total fatalities, and the number of unlocated records for that state.

Clicking a state transitions to a county-level drill-down view. County boundaries are rendered as a choropleth colored by accident density using a threshold scale with six color buckets. Individual accident dots are rendered on an HTML5 canvas element overlaid on the SVG county paths. This separation ensures that the canvas dots do not interfere with SVG hover event detection on the county polygons, providing instant tooltip response regardless of how many dots are rendered. A clip path derived from the merged state boundary prevents dots from rendering outside the state outline. A back button returns to the national spike map view.

County accident counts are derived from the pre-computed CountyFIPS field in the CSV, requiring zero runtime geometry. When a state has unlocated records (state-centroid coordinates), county hover counts are prefixed with a tilde (~) to indicate the values are approximate minimums.

### Filter System

Eight filter dimensions are available, all operating simultaneously with AND logic between groups and OR logic within groups:

- **Year Range** with dual-handle slider, numeric input fields, decade shortcut buttons, and a reset button
- **Injury Severity** toggle buttons (Fatal, Serious, Minor, None, Unknown) in the sidebar legend
- **Weather Conditions** (VMC, VFR, IMC, IFR, Other)
- **Engine Configuration** (Single Engine, Multi Engine)
- **Aircraft Type** (Airplane, Helicopter, Glider, Balloon, Gyroplane, Ultralight, Weight-Shift, Powered Parachute, Unmanned, Blimp, Experimental, ATC, Commercial Spaceflight)
- **Purpose of Flight** (Personal, Instructional, Business, Aerial Application, Ferry, Executive, Other)
- **FAR Part** (91, 135, 121, 107 UAS, 103 Ultralight, 137 Agricultural, Other)
- **Multi-Aircraft Only** standalone toggle that activates all other filters and restricts display to the 2,124 flagged multi-aircraft events

All filters update every visualization in real time including the map, charts, sparklines, statistics, choropleth, spike map, and county drill-down. Chip filters support single-click toggle and double-click isolate (deactivates all others in the group).

### Analytics Drawer

A slide-out analytics drawer contains four D3.js visualizations at full size, all updating in real time with the current filtered dataset:

- **US State Choropleth** with threshold color scale, D3 custom tooltips showing state name, accident count, percentage, fatal count, fatality rate, and fatalities. State outlines highlight on hover.
- **Accidents Per Year** area/line chart with gradient fill, grid lines, and 5-year axis tick intervals.
- **Top 10 Aircraft Makes by Accident** horizontal bar chart with Plasma color scale.
- **Injury Severity Breakdown** donut chart with percentage labels and centered total count.

### Sidebar Visualizations

The sidebar header contains four sparkline SVG charts showing year-over-year trends for total accidents, fatal accidents, serious injuries, and fatalities. These update with every filter change providing an immediate visual indicator of whether a trend is improving or worsening for the selected filter combination.

Small versions of the year chart and makes chart are also rendered in the sidebar scroll area for quick reference without opening the analytics drawer.

### Responsive Design

The layout adapts across screen sizes using CSS media queries at four breakpoints. The sidebar narrows from 340px to 260px on smaller screens. The analytics and help drawers scale their width using calc() based on viewport width. On mobile devices the drawers render as full-screen overlays above the sidebar. All D3 charts use their container's clientWidth for responsive sizing.

## Challenges

**NTSB Data Inconsistency:** The most significant challenge was the inconsistency of the source data across six decades. Aircraft make names appeared in multiple formats (CESSNA, Cessna, cessna). The NumberOfEngines field contained blanks, zeros, and comma-separated values for multi-aircraft events. Probable cause narratives used varied phrasing for the same causal factors. Builder names with commas in the Make field (e.g., “ANKERMAN, DONALD L.”) were initially misidentified as multi-aircraft records. Each inconsistency required detection, analysis, and a targeted fix.

**Canvas vs DOM Rendering:** Leaflet's preferCanvas option caused cluster markers rendered as circleMarkers on a canvas element to lose click interactivity. Switching cluster markers to DOM-based divIcon elements resolved the click issue but increased rendering time due to creating 168,793 individual DOM elements. The All Dots mode uses a dedicated canvas renderer that must be properly destroyed (not just hidden) when switching modes to prevent the canvas from intercepting click events on other layers.

**Cluster Rendering Performance:** Building 168,793 MarkerCluster markers takes several seconds even on fast hardware. A batched rendering system with a progress bar and disabled mode buttons was implemented to prevent users from clicking away during the build process. A requestAnimationFrame polling mechanism detects when cluster icons actually appear in the DOM before dismissing the loading indicator, ensuring consistent behavior regardless of machine speed. An edge case where zero filtered records caused the polling to run indefinitely was addressed with an early exit check.

**County Drill-Down Performance:** Initial implementations performed point-in-polygon geometry checks at runtime for each accident against all county polygons when a state was clicked. This took 4+ seconds for large states like Texas and California. Moving the county assignment to offline preprocessing and storing the results in the CSV eliminated all runtime geometry, making county views render instantly.

**County Hover Lag:** Rendering thousands of SVG circle elements for accident dots caused severe hover lag on county polygons due to browser hit-testing through the DOM tree. Replacing SVG circles with a single HTML5 canvas element for dot rendering eliminated the lag entirely while maintaining SVG-based interactive county

hover tooltips.

## Limitations

- Approximately 14,658 records have only state-level coordinate accuracy, placed at state geographic centroids. These records are included in all statistical calculations but their map positions are approximate.
- Only 679 of the 14,658 merged records have probable cause text. Most are older records (1960s–1970s) where the NTSB did not digitize the probable cause narrative.
- The application is entirely client-side. The 37MB CSV file must be downloaded in full before the application becomes functional. On slow internet connections this initial load can take 30–60 seconds, though GitHub Pages serves the file with gzip compression reducing the actual transfer to approximately 10MB.
- Cluster mode rendering time is dependent on client hardware. Older machines may experience 15–30 second rendering times when switching to cluster mode with the full unfiltered dataset.
- County-level accident counts for states with unlocated records are minimum values, not exact totals.

## Discussion

The project demonstrates that a single-page browser application with no backend infrastructure can effectively visualize and enable interactive exploration of a large-scale aviation safety dataset. The combination of Leaflet for geographic visualization and D3.js for analytical charts provides complementary strengths. Leaflet handles the interactive pan/zoom map with efficient tile rendering and marker management. D3 provides the precise control needed for custom chart types including the spike map, choropleth, and donut chart.

The data preprocessing pipeline proved to be as significant as the visualization implementation itself. The raw NTSB data required coordinate recovery, standardization of inconsistent fields, flagging of multi-aircraft events, and geographic enrichment with county-level assignments. Without this preprocessing, the visualization would have been limited to plotting raw coordinates with no meaningful filtering or geographic aggregation.

The spike map with county drill-down provides a visualization capability not available in the NTSB's own query tools. Users can identify state-level patterns, drill into county-level density, and cross-reference with filters for aircraft type, weather conditions, and injury severity to investigate specific scenarios.

Future work on this project could include automated classification of probable cause narratives using natural language processing or fine-tuned language models. A pre-computed cause category field would enable real-time analysis of why accidents happen for any filtered combination, transitioning the tool from a visualization platform to an analytical one. Integration of a small fine-tuned language model running locally via WebLLM could enable conversational queries against the dataset directly within the application.

## Conclusion

The project integrates geographic visualization, analytical charting, and multi-dimensional filtering into a single interactive platform for exploring NTSB aviation accident data. Six D3.js visualization types (line chart, bar chart, donut chart, sparklines, choropleth, and spike map) work alongside three Leaflet map modes (cluster, heatmap, all dots) and a timeline animation to provide multiple perspectives on the same dataset. All visualizations update in real time as filters are applied. The spike map with state-to-county drill-down enables geographic analysis from national overview to individual county level. The entire application runs client-side as a static site deployable on GitHub Pages with no server dependencies.

From choosing a visualization approach, cleaning and enriching 168,793 records of inconsistent government data, implementing multiple rendering strategies to handle performance constraints, and building an interactive analytical tool from start to finish as an individual teaches the integration of multiple JavaScript libraries, the challenges of working with real-world messy data, and the importance of preprocessing in any data visualization pipeline.

## Resources

- M1 MacBook Pro
- Visual Studio Code
- Python 3 with Pandas and Shapely for data cleaning and geographic preprocessing
- Leaflet 1.9.4 with MarkerCluster and Leaflet.heat plugins
- D3.js 7.8.5
- TopoJSON Client 3.1.0
- PapaParse 5.4.1
- noUiSlider 15.7.1
- CARTO basemap tiles (dark, voyager)
- FAA VFR Sectional chart tiles
- OpenTopoMap terrain tiles
- US Census Bureau state and county boundary data (TopoJSON format)
- NTSB Aviation Accident Database (public domain)

## References


- Leaflet documentation: https://leafletjs.com/
- D3.js documentation: https://d3js.org/
- MarkerCluster plugin: https://github.com/Leaflet/Leaflet.markercluster
- TopoJSON specification: https://github.com/topojson/topojson
- NTSB Aviation Accident Database: https://www.ntsb.gov/Pages/AviationQuery.aspx
- US Census Bureau geographic data: https://www.census.gov/geographies/mapping-files.html
- Chinnathambi, K. *JavaScript Absolute Beginner's Guide.*
- Haverbeke, M. *Eloquent JavaScript*, 3rd Edition.
- Wilson, K. *Web Development with HTML, CSS, and JavaScript.*
- Janert, P. K. *D3 for the Impatient: Interactive Graphics for Programmers & Scientists.*
- Meeks, E. *D3.js in Action.*
- Lubanovic, B. *Introducing Python: Modern Computing in Simple Packages.*
- Matthes, E. *Python Crash Course*, 3rd Edition.
- Pandas documentation: https://pandas.pydata.org/docs/
- Munzner, T. *Visualization Analysis and Design.*